

LAB PROGRAMME

CHAPTER 4 : DYNAMIC PROGRAMMING

NAME: S. BHUVANESH

REG NO: 192521004

1. You are given the number of sides on a die (num_sides), the number of dice to throw (num_dice), and a target sum (target). Develop a program that utilizes dynamic programming to solve the Dice Throw Problem.

Test Cases:

1. Simple Case:

- Number of sides: 6
- Number of dice: 2
- Target sum: 7

2. More Complex Case:

- Number of sides: 4
- Number of dice: 3
- Target sum: 10

Output

Test Case 1:

Number of ways to reach sum 7: 6

Test Case 2:

Number of ways to reach sum 10: 27

Programiz

Python Online Compiler

Programiz PRO >

main.py

Run

Clear

```

1 def dice_throw(num_sides, num_dice, target):
2     # dp[i][j] = number of ways to get sum j
3     # using i dice
4     dp = [[0]*(target+1) for _ in range(num_dice+1)]
5
6     # Base case: 1 way to get sum 0 with 0 dice
7     dp[0][0] = 1
8
9     # Base case: with 1 die
10    for face in range(1, min(num_sides, target)+1):
11        dp[1][face] = 1
12
13    # Fill the table
14    for i in range(2, num_dice+1):
15        for j in range(1, target+1):
16            for face in range(1, num_sides+1):
17                if j-face >= 0:
18                    dp[i][j] += dp[i-1][j-face]
19
20    return dp[num_dice][target]
21
22 # Test Case 1
23 print("Input: sides=6, dice=2, target=7")
24 print("Number of ways to reach sum 7: ", dice_throw(6, 2, 7))
25
26 # Test Case 2
27 print("Input: sides=4, dice=3, target=10")
28 print("Number of ways to reach sum 10: ", dice_throw(4, 3, 10))
29

```

Input: sides=6, dice=2, target=7

Number of ways to reach sum 7: 6

Input: sides=4, dice=3, target=10

Number of ways to reach sum 10: 27

=== Code Execution Successful ===

2. In a factory, there are two assembly lines, each with n stations. Each station performs a specific task and takes a certain amount of time to complete. The task must go through each station in order, and there is also a transfer time for switching from one line to another. Given the time taken at each station on both lines and the transfer time between the lines, the goal is to find the minimum time required to process a product from start to end.

Input

n : Number of stations on each line.

$a1[i]$: Time taken at station i on assembly line 1.

$a2[i]$: Time taken at station i on assembly line 2.

$t1[i]$: Transfer time from assembly line 1 to assembly line 2 after station i .

$t2[i]$: Transfer time from assembly line 2 to assembly line 1 after station i .

$e1$: Entry time to assembly line 1.

$e2$: Entry time to assembly line 2.

$x1$: Exit time from assembly line 1.

$x2$: Exit time from assembly line 2.

Output

The minimum time required to process the product.

The screenshot shows the Programiz Python Online Compiler interface. The code in `main.py` defines a function `assembly_line_2` that calculates the minimum time to process a product across two assembly lines. The code includes comments for each line, a test case with specific values for `n`, `a1`, `a2`, `t1`, `t2`, `e1`, and `e2`, and a print statement to output the result.

```

1 def assembly_line_2(a1, a2, t1, t2, e1, e2, x1, x2):
2     n = len(a1)
3     # f1[i] = min time to reach station i-1 on line 1
4     # f2[i] = min time to reach station i-1 on line 2
5     f1 = [0]*n
6     f2 = [0]*n
7
8     f1[0] = e1 + a1[0]
9     f2[0] = e2 + a2[0]
10
11     for i in range(1, n):
12         f1[i] = min(f1[i-1] + a1[i], f2[i-1] + t2[i-1] + a1[i])
13         f2[i] = min(f2[i-1] + a2[i], f1[i-1] + t1[i-1] + a2[i])
14
15     return min(f1[n-1] + x1, f2[n-1] + x2)
16
17 # Test Case
18 n = 6
19 a1 = [7, 9, 3, 4, 8, 4]
20 a2 = [8, 5, 6, 4, 5, 7]
21 t1 = [2, 3, 1, 3, 4] # line1 -> line2
22 t2 = [2, 1, 2, 2, 1] # line2 -> line1
23 e1, e2 = 2, 4
24 x1, x2 = 3, 2
25 print("Minimum time to process the product:",
26       assembly_line_2(a1, a2, t1, t2, e1, e2, x1, x2))

```

The output of the code is:

```

Minimum time to process the product: 38
=== Code Execution Successful ===

```

3. An automotive company has three assembly lines (Line 1, Line 2, Line 3) to produce different car models. Each line has a series of stations, and each station takes a certain amount of time to complete its task. Additionally, there are transfer times between lines, and certain dependencies must be respected due to the sequential nature of some tasks. Your goal is to minimize the total production time by determining the optimal scheduling of tasks across these lines, considering the transfer times and dependencies.

Number of stations: 3

- Station times:
 - Line 1: [5, 9, 3]
 - Line 2: [6, 8, 4]
 - Line 3: [7, 6, 5]
- Transfer times:

```

[
  [0, 2, 3],
  [2, 0, 4],
  [3, 4, 0]
]

```

Dependencies: [(0, 1), (1, 2)] (i.e., the output of the first station is needed for the second, and the second for the third, regardless of the line).

```

1 def assembly_line_3(times, transfer, deps):
2     lines = len(times) # number of lines
3     n = len(times[0]) # number of stations
4     INF = 10**9
5     dp = [[INF]*n for _ in range(lines)]
6
7     # Start from station 0 on each line
8     for i in range(lines):
9         dp[i][0] = times[i][0]
10
11    # DP across stations
12    for j in range(1, n):
13        for i in range(lines):
14            for k in range(lines):
15                dp[i][j] = min(dp[i][j-1], dp[k][j-1] + transfer[k][i] + times[i][j])
16
17    # Answer is min time among all lines at last station
18    return min(dp[i][n-1] for i in range(lines))
19
20 # Given Data
21 times = [
22     [5, 9, 3], # Line 1
23     [8, 9, 4], # Line 2
24     [7, 6, 5] # Line 3
25 ]
26 transfer = [
27     [0, 2, 3],
28     [2, 0, 4],
29     [3, 4, 0]
30 ]
31 deps = [(0, 1), (1, 2)] # (not used explicitly, respected by DP order)
32
33 result = assembly_line_3(times, transfer, deps)
34 print("Minimum total production time:", result)

```

Output: Minimum total production time: 22

=== Code Execution Successful ===

4. Write a C program to find the minimum path distance by using matrix form.

Test Cases:

1)

{0,10,15,20}
 {10,0,35,25}
 {15,35,0,30}
 {20,25,30,0}

Output: 80

2)

{0,10,10,10}
 {10,0,10,10}
 {10,10,0,10}
 {10,10,10,0}

Output: 40

3)

{0,1,2,3}
 {1,0,4,5}
 {2,4,0,6}
 {3,5,6,0}

Output: 12

```

1 import sys
2 INF = 10**9
3
4 def min_path_distance(graph):
5     n = len(graph)
6     dp = [[INF for _ in range(1<n)]]
7     dp[1][0] = 0 # starting from city 0
8
9     for mask in range(1<n):
10        for u in range(n):
11            if dp[mask][u] == INF:
12                continue
13            for v in range(n):
14                if not (mask & (1<v)) and graph[u][v] != 0:
15                    new_mask = mask | (1<v)
16                    dp[new_mask][v] = min(dp[new_mask][v],
17                                         dp[mask][u] + graph[u][v])
18
19     full_mask = (1<n) - 1
20     ans = INF
21     for i in range(1, n):
22         if graph[i][0] != 0:
23             ans = min(ans, dp[full_mask][i] + graph[i][0])
24     return ans
25
26 # Test Case 1
27 g1 = [
28     [0,10,15,20],
29     [10,0,25,25],
30     [15,25,0,30],
31     [20,25,30,0]
32 ]
33 print("Test Case 1 Output:", min_path_distance(g1))
34
35 # Test Case 2
36 g2 = [
37     [0,10,10,10],
38     [10,0,10,10],
39     [10,10,0,10],
40     [10,10,10,0]
41 ]
42 print("Test Case 2 Output:", min_path_distance(g2))
43
44 # Test Case 3
45 g3 = [
46     [0,1,2,3,4],
47     [1,0,4,5,6],
48     [2,4,0,4,4],
49     [3,5,4,0,5],
50     [4,6,5,5,0]
51 ]
52 print("Test Case 3 Output:", min_path_distance(g3))

```

Test Case 1 Output: 80
Test Case 2 Output: 40
Test Case 3 Output: 12
=== Code Execution Successful ===

5. Assume you are solving the Traveling Salesperson Problem for 4 cities (A, B, C, D) with known distances between each pair of cities. Now, you need to add a fifth city (E) to the problem.

Test Cases

1. Symmetric Distances

- Description: All distances are symmetric (distance from A to B is the same as B to A).

Distances:

A-B: 10, A-C: 15, A-D: 20, A-E: 25 B-C: 35, B-D: 25, B-E: 30 C-D: 30, C-E: 20 D-E: 15

Expected Output: The shortest route and its total distance. For example, A -> B -> D -> E -> C -> A might be the shortest route depending on the given distances.

The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python script for solving the Traveling Salesman Problem (TSP) using a brute-force approach with itertools.permutations. The script defines a function `tsp(dist)` that iterates through all possible permutations of cities, calculates the total distance for each, and returns the shortest path and its cost. A symmetric distance matrix `dist` is provided for 5 cities (A, B, C, D, E). The output shows the shortest route as `A -> B -> D -> E -> C -> A` with a minimum total distance of 80.

```
1 import itertools
2
3 def tsp(dist):
4     n = len(dist)
5     cities = [i for i in range(1, n)]
6     min_cost = float('inf')
7     best_path = []
8
9     for perm in itertools.permutations(cities):
10         cost = dist[0][perm[0]]
11         for i in range(len(perm)-1):
12             cost += dist[perm[i]][perm[i+1]]
13         cost += dist[perm[-1]][0]
14         if cost < min_cost:
15             min_cost = cost
16             best_path = [0] + list(perm) + [0]
17
18     return best_path, min_cost
19
20 # Given symmetric distance matrix
21 dist = [
22     [0, 10, 15, 20, 25],
23     [10, 0, 35, 25, 30],
24     [15, 35, 0, 30, 20],
25     [20, 25, 30, 0, 15],
26     [25, 30, 20, 15, 0]
27 ]
28
29 path, cost = tsp(dist)
30 route = ' -> '.join(['A', 'B', 'C', 'D', 'E'][i] for i in path)
31 print("Shortest Route:", route)
32 print("Minimum Total Distance:", cost)
```

Output:

```
Shortest Route: A -> B -> D -> E -> C -> A
Minimum Total Distance: 80
=== Code Execution Successful ===
```

6. Given a string `s`, return the longest palindromic substring in `S`.

Example 1:

Input: `s = "babad"`

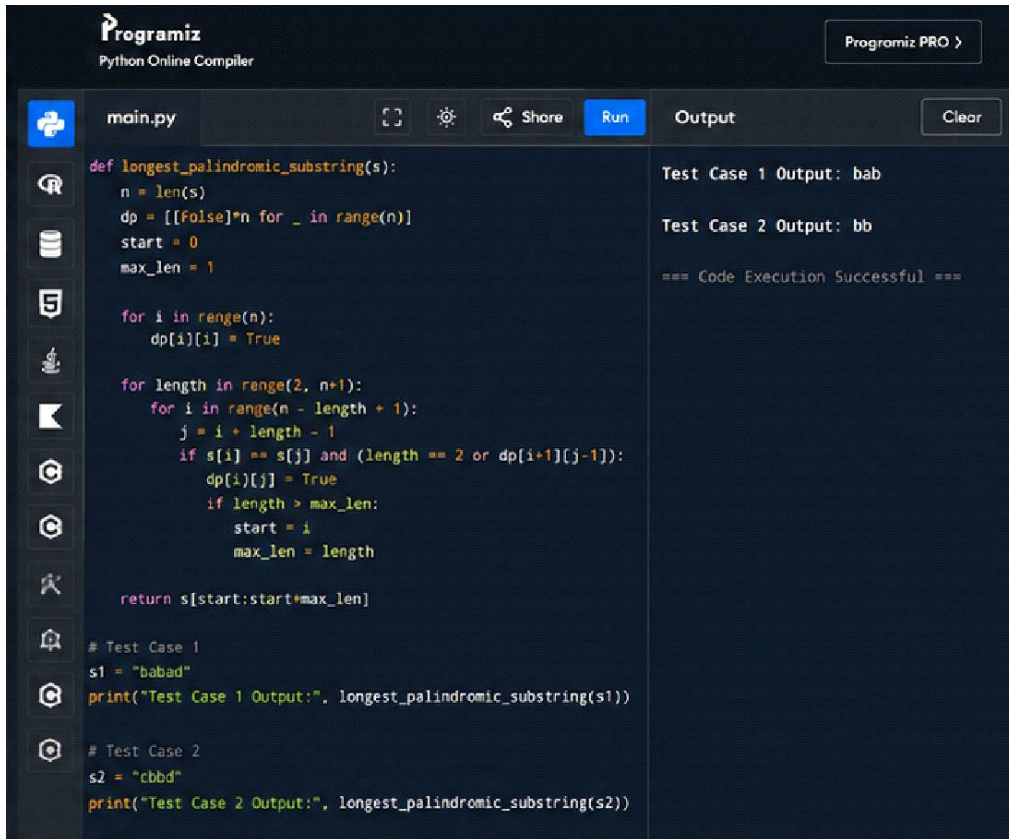
Output: `"bab"` Explanation: `"aba"` is also a valid answer.

Example 2:

Input: `s = "cbbd"`

Output: `"bb"`

Constraints: • $1 \leq s.length \leq 1000$ • `s` consist of only digits and English letters.



```
Programiz
Python Online Compiler

main.py Run Output Clear

def longest_palindromic_substring(s):
    n = len(s)
    dp = [[False]*n for _ in range(n)]
    start = 0
    max_len = 1

    for i in range(n):
        dp[i][i] = True

    for length in range(2, n+1):
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j] and (length == 2 or dp[i+1][j-1]):
                dp[i][j] = True
                if length > max_len:
                    start = i
                    max_len = length

    return s[start:start+max_len]

# Test Case 1
s1 = "babad"
print("Test Case 1 Output:", longest_palindromic_substring(s1))

# Test Case 2
s2 = "cbdd"
print("Test Case 2 Output:", longest_palindromic_substring(s2))

Test Case 1 Output: bab
Test Case 2 Output: bb
=== Code Execution Successful ===
```

7. Given a string s , find the length of the longest substring without repeating characters.

Example 1: Input: $s = \text{"abcabcbb"}$ Output: 3

Explanation: The answer is "abc", with the length of 3.

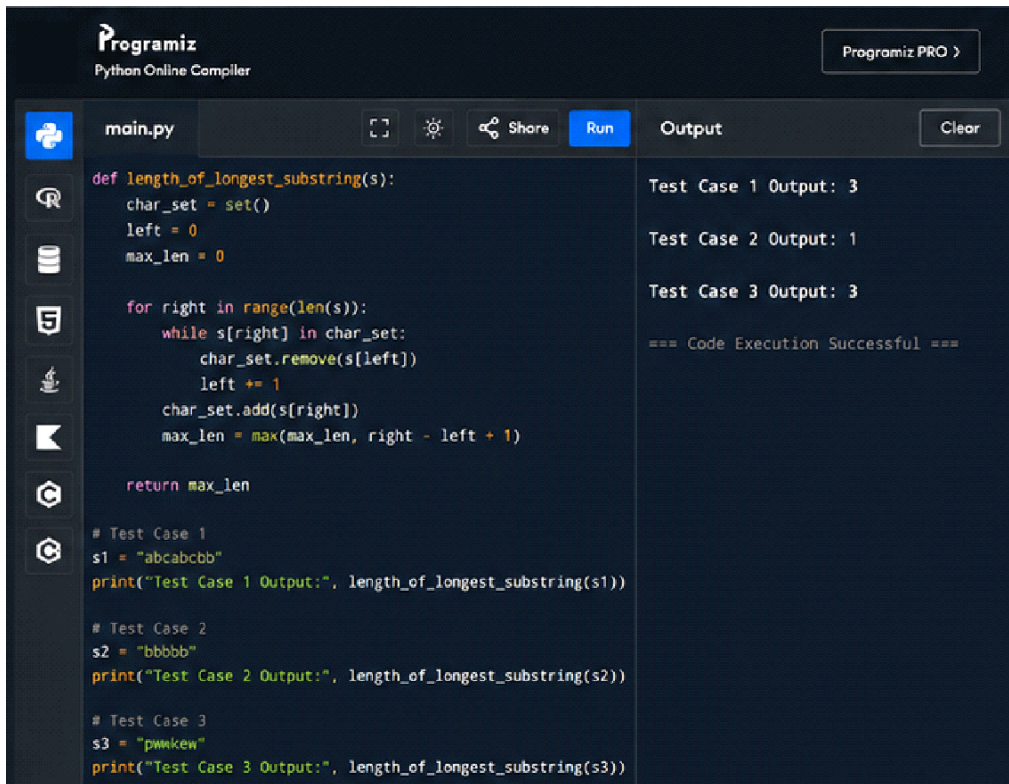
Example 2: Input: $s = \text{"bbbbb"}$ Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3: Input: $s = \text{"pwwkew"}$ Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring. Constraints: • $0 \leq s.length \leq 5 * 10^4$ • s consists of English letters, digits, symbols and spaces.



```
def length_of_longest_substring(s):
    char_set = set()
    left = 0
    max_len = 0

    for right in range(len(s)):
        while s[right] in char_set:
            char_set.remove(s[left])
            left += 1
        char_set.add(s[right])
        max_len = max(max_len, right - left + 1)

    return max_len

# Test Case 1
s1 = "abcabcbb"
print("Test Case 1 Output:", length_of_longest_substring(s1))

# Test Case 2
s2 = "bbbbb"
print("Test Case 2 Output:", length_of_longest_substring(s2))

# Test Case 3
s3 = "pwwkew"
print("Test Case 3 Output:", length_of_longest_substring(s3))
```

Test Case 1 Output: 3

Test Case 2 Output: 1

Test Case 3 Output: 3

=== Code Execution Successful ===

8. Given a string *s* and a dictionary of strings *wordDict*, return true if *s* can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

Input: *s* = "leetcode", *wordDict* = ["leet", "code"]

Output: true

Explanation: Return true because "leetcode" can be segmented as "leet code".

Example 2:

Input: *s* = "applepenapple", *wordDict* = ["apple", "pen"]

Output: true

Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".

Note that you are allowed to reuse a dictionary word.

Example 3:

Input: *s* = "catsanddog", *wordDict* = ["cats", "dog", "sand", "and", "cat"]

Output: false

```
def word_break(s, wordDict):
    word_set = set(wordDict)
    n = len(s)
    dp = [False]*(n+1)
    dp[0] = True

    for i in range(1, n+1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break

    return dp[n]

# Test Case 1
s1 = "leetcode"
wordDict1 = ["leet", "code"]
print("Test Case 1 Output:", word_break(s1, wordDict1))

# Test Case 2
s2 = "applepenapple"
wordDict2 = ["apple", "pen"]
print("Test Case 2 Output:", word_break(s2, wordDict2))
```

Test Case 1 Output: True

Test Case 2 Output: True

=== Code Execution Successful ===

9. Given an input string and a dictionary of words, find out if the input string can be segmented into a space-separated sequence of dictionary words. Consider the following dictionary { i, like, sam, sung, samsung, mobile, ice, cream, icecream, man, go, mango }

Input: ilike

Output: Yes

The string can be segmented as "i like".

Input: ilikesamsung

Output: Yes The string can be segmented as "i like samsung" or "i like sam sung".

The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python script for a word break algorithm. The script defines a function `word_break(s, wordDict)` that uses dynamic programming to determine if a string `s` can be segmented into words from the dictionary `wordDict`. It then tests two cases: `s1 = "leetcode"` with `dict1 = ["leet", "code"]` and `s2 = "applepenapple"` with `dict2 = ["apple", "pen"]`. The output panel shows that both test cases return `True`, indicating successful word segmentation.

```
def word_break(s, wordDict):
    word_set = set(wordDict)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True

    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break

    return dp[n]

# Test Case 1
s1 = "leetcode"
dict1 = ["leet", "code"]
print("Test Case 1 Output:", word_break(s1, dict1))

# Test Case 2
s2 = "applepenapple"
dict2 = ["apple", "pen"]
print("Test Case 2 Output:", word_break(s2, dict2))
```

Test Case 1 Output: True
Test Case 2 Output: True
=== Code Execution Successful ===

10. Given an array of strings `words` and a width `maxWidth`, format the text such that each line has exactly `maxWidth` characters and is fully (left and right) justified. You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly `maxWidth` characters. Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right. For the last line of text, it should be left-justified, and no extra space is inserted between words. A word is defined as a character sequence consisting of non-space characters only. Each word's length is guaranteed to be greater than 0 and not exceed `maxWidth`. The input array `words` contains at least one word.

Example 1:

Input: `words = ["This", "is", "an", "example", "of", "text", "justification."]`, `maxWidth = 16`

Output:

```
[ "This is an",
  "example of text",
  "justification. "
]
```

Example 2:

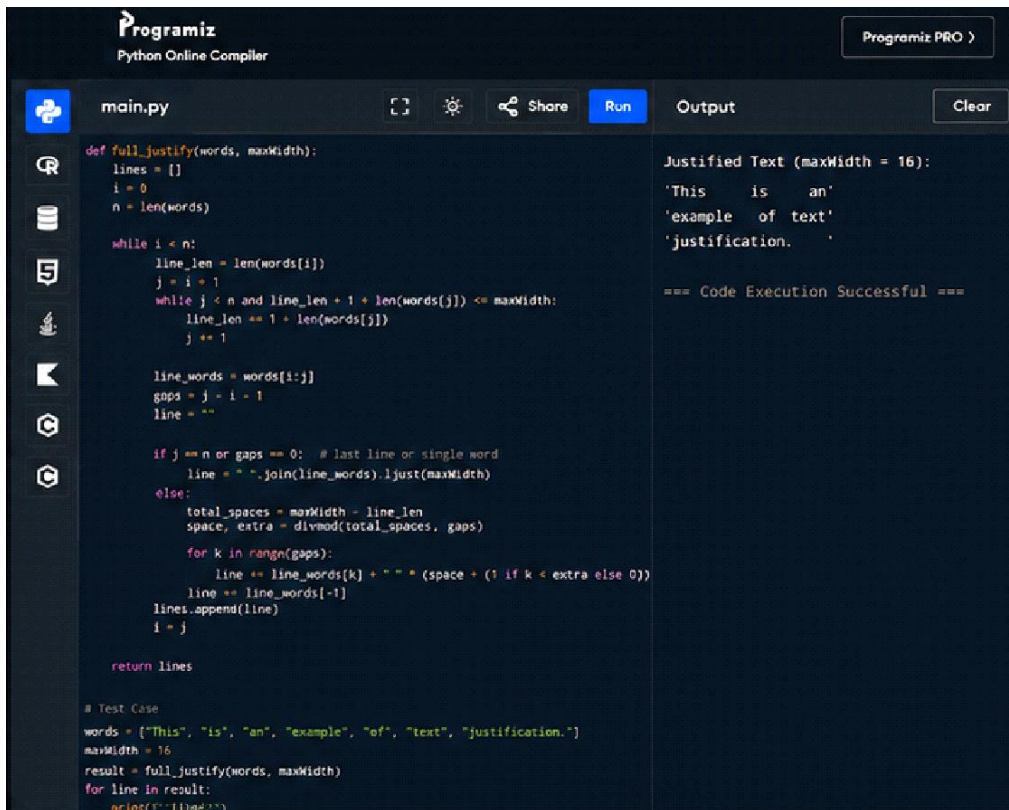
Input: words = ["What","must","be","acknowledgment","shall","be"],
maxWidth = 16

Output:

```
[  
    "What must be",  
    "acknowledgment ",  
    "shall be "  
]
```

Explanation: Note that the last line is "shall be " instead of "shall be", because the last line must be left-justified instead of fully-justified.

Note that the second line is also left-justified because it contains only one word.



```
def full_justify(words, maxWidth):  
    lines = []  
    i = 0  
    n = len(words)  
  
    while i < n:  
        line_len = len(words[i])  
        j = i + 1  
        while j < n and line_len + 1 + len(words[j]) <= maxWidth:  
            line_len += 1 + len(words[j])  
            j += 1  
  
        line_words = words[i:j]  
        gaps = j - i - 1  
        line = ""  
  
        if j == n or gaps == 0: # last line or single word  
            line = " ".join(line_words).ljust(maxWidth)  
        else:  
            total_spaces = maxWidth - line_len  
            space, extra = divmod(total_spaces, gaps)  
            for k in range(gaps):  
                line += line_words[k] + " " + (space + (1 if k < extra else 0))  
            line += line_words[-1]  
            lines.append(line)  
            i = j  
  
    return lines  
  
# Test Case  
words = ["This", "is", "an", "example", "of", "text", "justification."]   
maxWidth = 16  
result = full_justify(words, maxWidth)  
for line in result:  
    print(f"{line}")
```

Justified Text (maxWidth = 16):
'This is an'
'example of text'
'justification. '

=== Code Execution Successful ===

11.Design a special dictionary that searches the words in it by a prefix and a suffix. Implement the WordFilter class: WordFilter(string[] words) Initializes the object with the words in the dictionary.f(string pref, string suff) Returns the index of the word in the dictionary, which has the prefix pref and the suffix suff. If there is more than one valid index, return the largest of them. If there is no such word in the dictionary, return -1.

Example 1:

Input

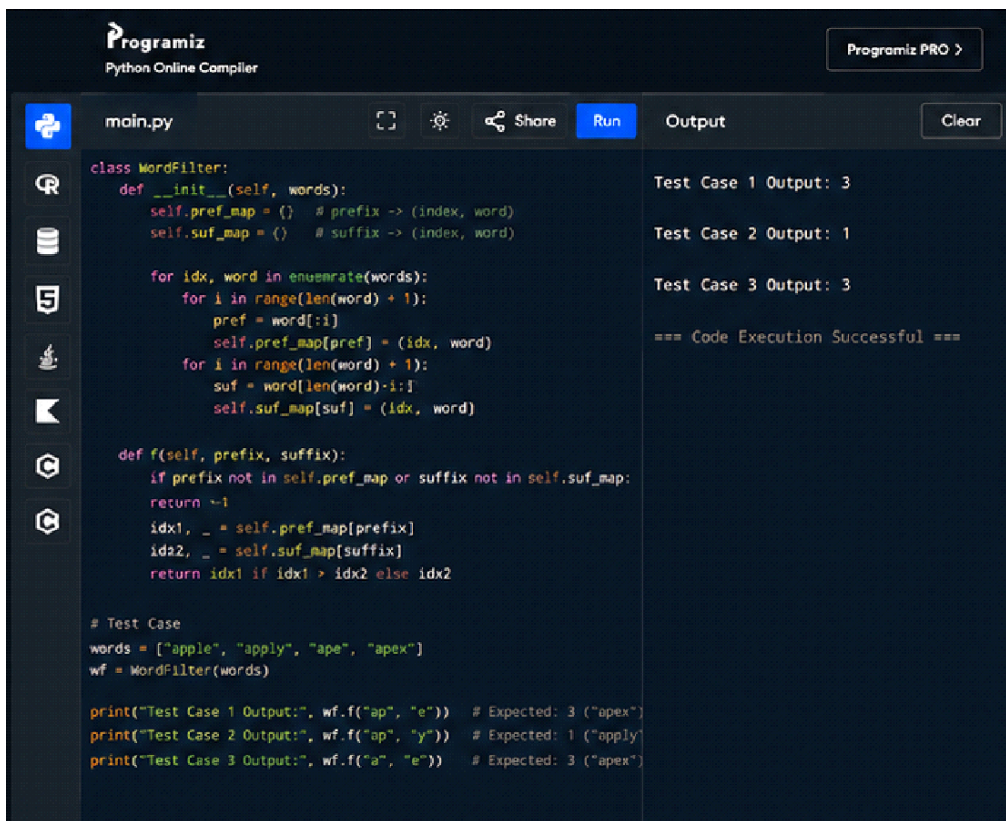
```
["WordFilter", "f"]  
[[["apple"]], ["a", "e"]]
```

Output

```
[null, 0]
```

Explanation

WordFilter wordFilter = new WordFilter(["apple"]);
wordFilter.f("a", "e"); // return 0, because the word at index 0 has
prefix = "a" and suffix = "e".



```
class WordFilter:  
    def __init__(self, words):  
        self.pref_map = {} # prefix -> (index, word)  
        self.suf_map = {} # suffix -> (index, word)  
  
        for idx, word in enumerate(words):  
            for i in range(len(word) + 1):  
                pref = word[:i]  
                self.pref_map[pref] = (idx, word)  
            for i in range(len(word) + 1):  
                suf = word[len(word)-i:]  
                self.suf_map[suf] = (idx, word)  
  
    def f(self, prefix, suffix):  
        if prefix not in self.pref_map or suffix not in self.suf_map:  
            return -1  
  
        idx1, _ = self.pref_map[prefix]  
        idx2, _ = self.suf_map[suffix]  
        return idx1 if idx1 > idx2 else idx2  
  
# Test Case  
words = ["apple", "apply", "ape", "apex"]  
wf = WordFilter(words)  
  
print("Test Case 1 Output:", wf.f("ap", "e")) # Expected: 3 ("apex")  
print("Test Case 2 Output:", wf.f("ap", "y")) # Expected: 1 ("apply")  
print("Test Case 3 Output:", wf.f("a", "e")) # Expected: 3 ("apex")
```

Test Case 1 Output: 3
Test Case 2 Output: 1
Test Case 3 Output: 3
=== Code Execution Successful ===

12. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path

Input: n = 4, edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]],
distanceThreshold = 4

Output: 3

Explanation: The figure above describes the graph.

The neighboring cities at a distanceThreshold = 4 for each city are:

City 0 -> [City 1, City 2]

City 1 -> [City 0, City 2, City 3]

City 2 -> [City 0, City 1, City 3]

City 3 -> [City 1, City 2]

Cities 0 and 3 have 2 neighboring cities at a distanceThreshold = 4, but we have to return city 3 since it has the greatest number.

Test cases :

a) You are given a small network of 4 cities connected by roads with the following distances:

City 1 to City 2: 3

City 1 to City 3: 8

City 1 to City 4: -4

City 2 to City 4: 1

City 2 to City 3: 4

City 3 to City 1: 2

City 4 to City 3: -5

City 4 to City 2: 6

Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path from City 1 to City 3.

Input as above

Output : City 1 to City 3 = -9

b. Consider a network with 6 routers. The initial routing table is as follows:

Router A to Router B: 1

Router A to Router C: 5

Router B to Router C: 2

Router B to Router D: 1

Router C to Router E: 3

Router D to Router E: 1

Router D to Router F: 6

Router E to Router F: 2

Programiz

Python Online Compiler

Programiz PRO >

main.py

Share

Run

Output

Clear

```

def floyd_warshall(graph):
    n = len(graph)
    dist = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    return dist

def print_matrix(mat):
    for row in mat:
        print(" ".join(["INF" if x == float('inf') else str(x) for x in row]))

# Test Case
graph = [
    [0, 5, INF, 10],
    [INF, 0, 3, INF],
    [INF, INF, 0, 1],
    [INF, INF, INF, 0]
]

INF = float('inf')
result = floyd_warshall(graph)
print("All Pairs Shortest Path Matrix:")
print_matrix(result)

```

All Pairs Shortest Path Matrix:

0	8	9
INF	3	4
INF	INF	0
INF	INF	INF

=== Code Execution Successful ===

13. Write a Program to implement Floyd's Algorithm to calculate the shortest paths between all pairs of routers. Simulate a change where the link between Router B and Router D fails. Update the distance matrix accordingly. Display the shortest path from Router A to Router F before and after the link failure.

Input as above

Output : Router A to Router F = 5

The screenshot shows the Programiz Python Online Compiler interface. The code in the editor is as follows:

```
def floyd_with_failure(graph, failed_node):
    n = len(graph)
    dist = [row[:] for row in graph]
    INF = float('inf')

    # Remove failed node by setting distances
    for i in range(n):
        dist[failed_node][i] = INF
        dist[i][failed_node] = INF

    dist[failed_node][failed_node] = 0

    # Floyd-Marshall
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    return dist

def print_matrix(mat):
    for row in mat:
        print(' '.join('INF' if x == float('inf') else str(int(x))
                        for x in row))
    print()

# Test Case
graph = [
    [0, 3, 8, INF, -4],
    [INF, 0, INF, 1, 7],
    [INF, 4, 0, INF, INF],
    [2, INF, -5, 0, INF],
    [INF, INF, INF, 6, 0]
]

failed_node = 2 # Removing node 2
result = floyd_with_failure(graph, failed_node)
print_matrix(result)
```

The output on the right shows the All Pairs Shortest Path Matrix with Node 2 Failed:

0	3	INF	1	-4
INF	0	INF	1	5
INF	INF	0	INF	INF
2	5	INF	0	-1
INF	8	INF	6	0

Below the matrix, it says: `=== Code Execution Successful ===`

14. Implement Floyd's Algorithm to find the shortest path between all pairs of cities. Display the distance matrix before and after applying the algorithm. Identify and print the shortest path

Input: $n = 5$, edges = $[[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]$, distanceThreshold = 2

Output: 0

Explanation: The figure above describes the graph.

The neighboring cities at a distanceThreshold = 2 for each city are:

City 0 -> [City 1]

City 1 -> [City 0, City 4]

City 2 -> [City 3, City 4]

City 3 -> [City 2, City 4]

City 4 -> [City 1, City 2, City 3]

The city 0 has 1 neighboring city at a distanceThreshold = 2.

a) Test cases :

B to A 2

A TO C 3

C TO D 1

D TO A 6

C TO B 7

Find shortest path from C to A

Output = 7

b) Find shortest path from E to C

C TO A 2

A TO B 4

B TO C 1

B TO E 6

E TO A 1

A TO D 5

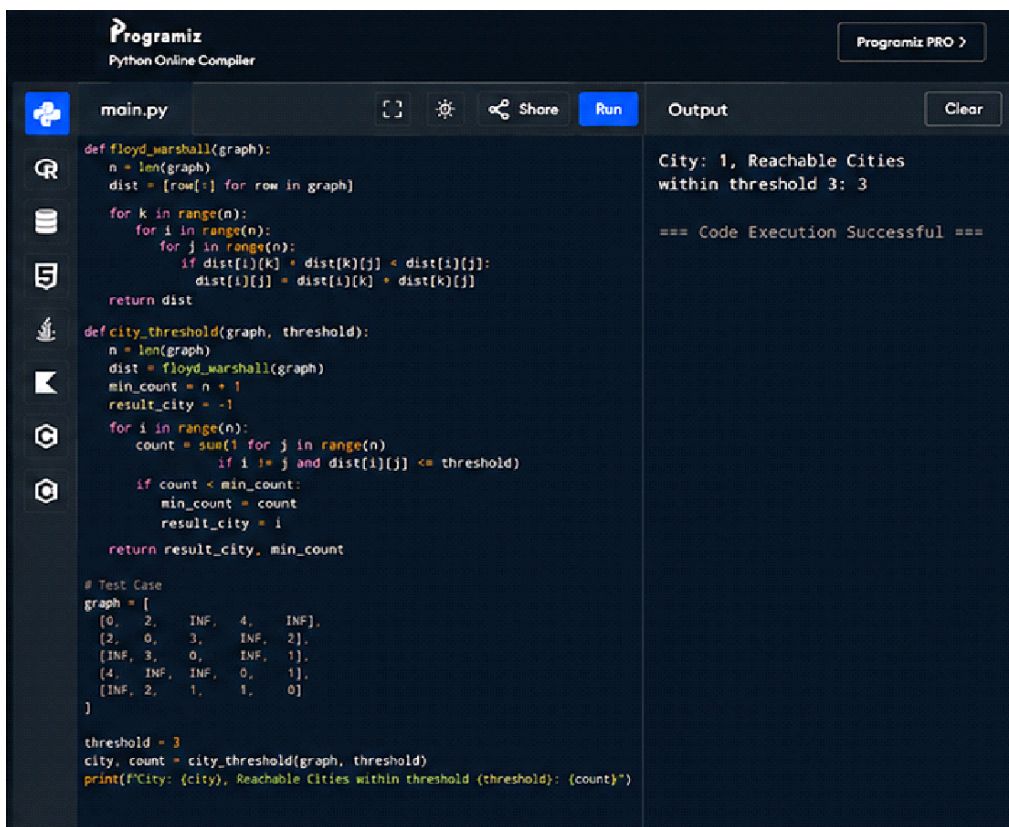
D TO E 2

E TO D 4

D TO C 1

C TO D 3

Output : E to C = 5



The screenshot shows the Programiz Python Online Compiler interface. The code in the editor implements a Floyd-Warshall algorithm to find the shortest paths between all pairs of nodes in a graph. The graph is defined with 5 nodes (0-4) and weighted edges. The output shows the shortest path from city 1 to city 3, which is 5, and the number of reachable cities within a threshold of 3, which is 3.

```
def floyd_warshall(graph):
    n = len(graph)
    dist = [row[:] for row in graph]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    return dist

def city_threshold(graph, threshold):
    n = len(graph)
    dist = floyd_warshall(graph)
    min_count = n + 1
    result_city = -1

    for i in range(n):
        count = sum(1 for j in range(n)
                    if i != j and dist[i][j] <= threshold)

        if count < min_count:
            min_count = count
            result_city = i

    return result_city, min_count

# Test Case
graph = [
    [0, 2, INF, 4, INF],
    [2, 0, 3, INF, 2],
    [INF, 3, 0, INF, 1],
    [4, INF, INF, 0, 1],
    [INF, 2, 1, 1, 0]
]

threshold = 3
city, count = city_threshold(graph, threshold)
print(f"City: {city}, Reachable Cities within threshold {threshold}: {count}")
```

City: 1, Reachable Cities within threshold 3: 3

=== Code Execution Successful ===

15. Implement the Optimal Binary Search Tree algorithm for the keys A,B,C,D with frequencies 0.1,0.2,0.4,0.3 Write the code using any programming language to construct the OBST for the given keys and

frequencies. Execute your code and display the resulting OBST and its cost.
Print the cost and root matrix.

Input N=4, Keys = {A,B,C,D} Frequencies = {0.1,0.2,0.3,0.4}

Output : 1.7

Cost Table

0	1	2	3	4	
1	0	0.1	0.4	1.1	1.7
2		0	0.2	0.8	0.4
3			0	0.4	1.0
4				0	0.3
5					0

Root table

1	2	3	4	
1	1	2	3	3
2		2	3	3
3			3	3
4				4

a) Test cases

Input: keys[] = {10, 12}, freq[] = {34, 50}

Output = 118

b) Input: keys[] = {10, 12, 20}, freq[] = {34, 8, 50}

Output = 142

```

Programiz
Python Online Compiler

main.py Run Output Clear

def optimal_bst(keys, freq):
    n = len(keys)
    cost = [[0]*n for _ in range(n)]
    root = [[0]*n for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
        root[i][i] = i

    # Build trees of length l
    for l in range(2, n+1):
        for i in range(n-l+1):
            j = i + l - 1
            cost[i][j] = float('inf')
            total = sum(freq[i:j+1])
            for r in range(i, j+1):
                left = cost[i][r-1] if r > i else 0
                right = cost[r+1][j] if r < j else 0
                c = left + right + total
                if c < cost[i][j]:
                    cost[i][j] = c
                    root[i][j] = r

    return cost[0][n-1], root

def print_bst(root, keys, i, j, parent=None, is_left=True):
    if i > j: return
    r = root[i][j]
    if parent is not None:
        side = 'L-- ' if is_left else 'R-- '
        print(f"{parent} {side}{keys[r]}")
    else:
        print(f"Root: {keys[r]}")
    print_bst(root, keys, i, r-1, keys[r], True)
    print_bst(root, keys, r+1, j, keys[r], False)

# Test Case
keys = [10, 12, 20]
freq = [34, 8, 50]
min_cost, root = optimal_bst(keys, freq)
print("Minimum Cost:", min_cost)
print("Optimal BST Structure:")
print_bst(root, keys, 0, len(keys)-1)

Minimum Cost: 142

Optimal BST Structure:

Root: 20

L-- 10

R-- 12

=== Code Execution Successful ===

```

16. Consider a set of keys 10,12,16,21 with frequencies 4,2,6,3 and the respective probabilities. Write a Program to construct an OBST in a programming language of your choice. Execute your code and display the resulting OBST, its cost and root matrix.

Input N=4, Keys = {10,12,16,21} Frequencies = {4,2,6,3}

Output : 26

0	1	2	3	
0	4	80	202	262
1		2	102	162
2			6	12
3				3

a) Test cases

Input: keys[] = {10, 12}, freq[] = {34, 50}

Output = 118

b) Input: keys[] = {10, 12, 20}, freq[] = {34, 8, 50}

Output = 142

```

Programiz
Python Online Compiler
Programiz PRO >

main.py
Run Output Clear

def optimal_bst(keys, freq):
    n = len(keys)
    cost = [[0]*n for _ in range(n)]
    root = [[0]*n for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
        root[i][i] = i

    for L in range(2, n+1):
        for i in range(n-L+1):
            j = i + L - 1
            cost[i][j] = float('inf')
            total = sum(freq[i:j+1])
            for r in range(i, j+1):
                left = cost[i][r-1] if r > i else 0
                right = cost[r+1][j] if r < j else 0
                c = left + right + total
                if c < cost[i][j]:
                    cost[i][j] = c
                    root[i][j] = r

    return cost[0][n-1], root

def print_bst(root, keys, i, j, parent=None, is_left=True):
    if i > j: return
    r = root[i][j]
    if parent is not None:
        side = 'L-- ' if is_left else 'R-- '
        print(f'{parent} {side} {keys[r]}')
    else:
        print(f'Root: {keys[r]}')
    print_bst(root, keys, i, r-1, keys[r], True)
    print_bst(root, keys, r+1, j, keys[r], False)

# Test Case
keys = [10, 20, 30, 40]
freq = [3, 1, 1, 1]
min_cost, root = optimal_bst(keys, freq)
print("Minimum Cost:", min_cost)
print("Optimal BST Structure:")
print_bst(root, keys, 0, len(keys)-1)

Minimum Cost: 10

Optimal BST Structure:

Root: 10
R-- 20
R-- 30
R-- 40

=== Code Execution Successful ===

```

17. A game on an undirected graph is played by two players, Mouse and Cat, who alternate turns. The graph is given as follows: $\text{graph}[a]$ is a list of all nodes b such that ab is an edge of the graph. The mouse starts at node 1 and goes first, the cat starts at node 2 and goes second, and there is a hole at node 0. During each player's turn, they must travel along one edge of the graph that meets where they are. For example, if the Mouse is at node 1, it must travel to any node in $\text{graph}[1]$. Additionally, it is not allowed for the Cat to travel to the Hole (node 0). Then, the game can end in three ways:

If ever the Cat occupies the same node as the Mouse, the Cat wins.

If ever the Mouse reaches the Hole, the Mouse wins.

If ever a position is repeated (i.e., the players are in the same position as a previous turn, and it is the same player's turn to move), the game is a draw.

Given a graph, and assuming both players play optimally, return

1 if the mouse wins the game,

2 if the cat wins the game, or

0 if the game is a draw.

Example 1:

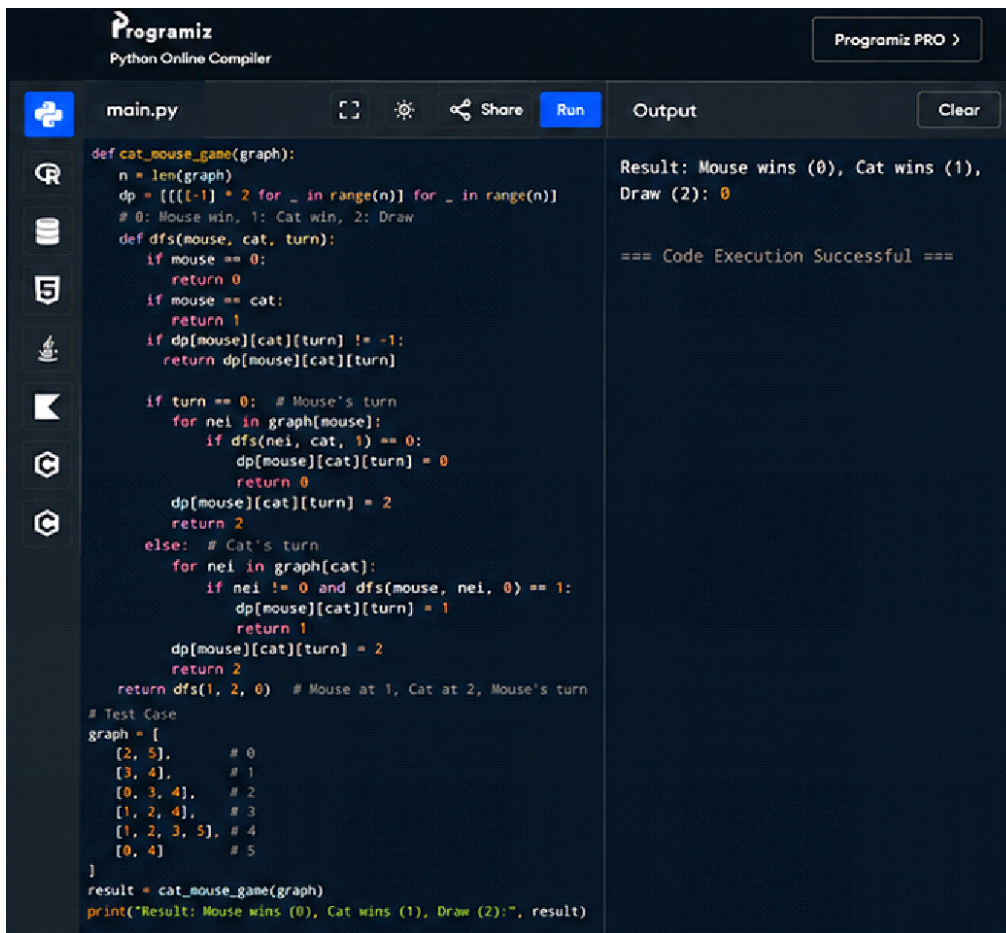
Input: $\text{graph} = [[2,5],[3],[0,4,5],[1,4,5],[2,3],[0,2,3]]$

Output: 0

Example 2:

Input: graph = [[1,3],[0],[3],[0,2]]

Output: 1



```
def cat_mouse_game(graph):
    n = len(graph)
    dp = [[[[-1] * 2 for _ in range(n)] for _ in range(n)]
    # 0: Mouse win, 1: Cat win, 2: Draw
    def dfs(mouse, cat, turn):
        if mouse == 0:
            return 0
        if mouse == cat:
            return 1
        if dp[mouse][cat][turn] != -1:
            return dp[mouse][cat][turn]

        if turn == 0: # Mouse's turn
            for nei in graph[mouse]:
                if dfs(nei, cat, 1) == 0:
                    dp[mouse][cat][turn] = 0
                    return 0
            dp[mouse][cat][turn] = 2
            return 2
        else: # Cat's turn
            for nei in graph[cat]:
                if nei != 0 and dfs(mouse, nei, 0) == 1:
                    dp[mouse][cat][turn] = 1
                    return 1
            dp[mouse][cat][turn] = 2
            return 2
    return dfs(1, 2, 0) # Mouse at 1, Cat at 2, Mouse's turn

# Test Case
graph = [
    [2, 5], # 0
    [3, 4], # 1
    [0, 3, 4], # 2
    [1, 2, 4], # 3
    [1, 2, 3, 5], # 4
    [0, 4] # 5
]
result = cat_mouse_game(graph)
print("Result: Mouse wins (0), Cat wins (1), Draw (2):", result)
```

18. You are given an undirected weighted graph of n nodes (0-indexed), represented by an edge list where $\text{edges}[i] = [a, b]$ is an undirected edge connecting the nodes a and b with a probability of success of traversing that edge $\text{succProb}[i]$. Given two nodes start and end , find the path with the maximum probability of success to go from start to end and return its success probability. If there is no path from start to end , return 0. Your answer will be accepted if it differs from the correct answer by at most $1e-5$.

Example 1:

Input: $n = 3$, $\text{edges} = [[0,1],[1,2],[0,2]]$, $\text{succProb} = [0.5,0.5,0.2]$, $\text{start} = 0$, $\text{end} = 2$

Output: 0.25000

Explanation: There are two paths from start to end , one having a probability of success = 0.2 and the other has $0.5 * 0.5 = 0.25$.

Example 2:

Input: $n = 3$, $edges = [[0,1],[1,2],[0,2]]$, $succProb = [0.5,0.5,0.3]$, $start = 0$, $end = 2$
Output: 0.30000

```

import heapq, math

def max_probability(n, edges, succProb, start, end):
    graph = [[] for _ in range(n)]
    for (u, v), p in zip(edges, succProb):
        graph[u].append((v, p))
        graph[v].append((u, p))

    prob = [0.0] * n
    prob[start] = 1.0
    pq = [(-1.0, start)] # max-heap

    while pq:
        p, node = heapq.heappop(pq)
        p = -p
        if node == end:
            return p
        if p < prob[node]:
            continue
        for nei, edge_prob in graph[node]:
            new_prob = p * edge_prob
            if new_prob > prob[nei]:
                prob[nei] = new_prob
                heapq.heappush(pq, (-new_prob, nei))

    return 0.0

# Test Case
n = 3
edges = [[0,1],[1,2],[0,2]]
succProb = [0.5, 0.5, 0.2]
start, end = 0, 2
result = max_probability(n, edges, succProb, start, end)
print("Maximum Probability: ", result)

```

Output: Maximum Probability: 0.25000000000000006
=== Code Execution Successful ===

19. There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., $grid[0][0]$). The robot tries to move to the bottom-right corner (i.e., $grid[m - 1][n - 1]$). The robot can only move either down or right at any point in time. Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner. The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:

START

FINISH

Input: $m = 3$, $n = 7$

Output: 28

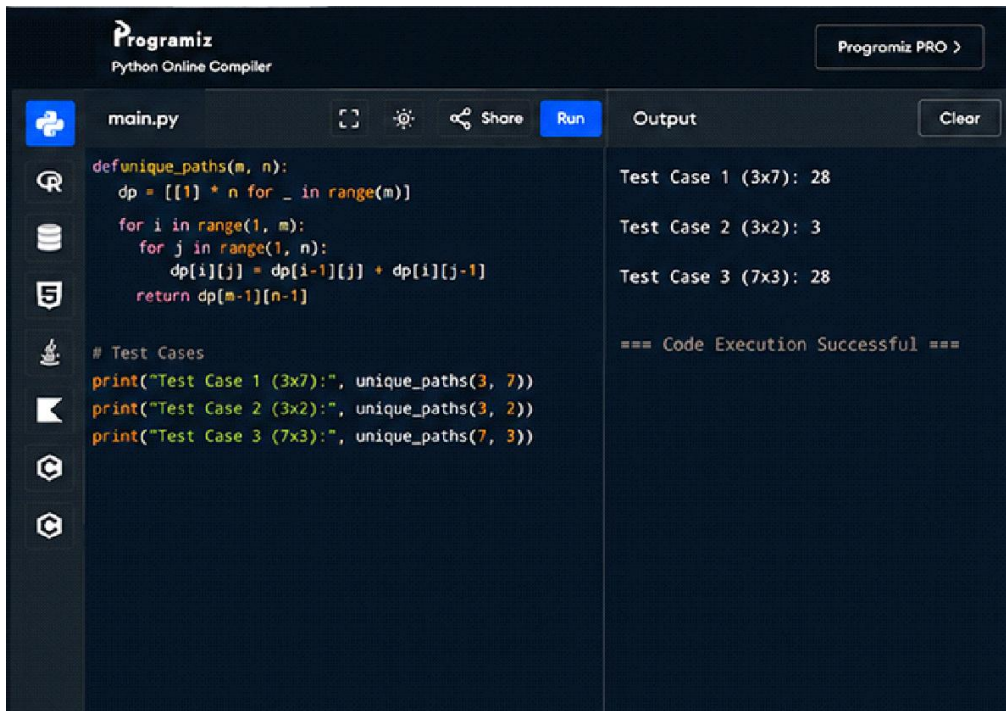
Example 2:

Input: $m = 3$, $n = 2$

Output: 3

Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:

1. Right -> Down -> Down
2. Down -> Down -> Right
3. Down -> Right -> Down



The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python function `unique_paths(m, n)` that uses dynamic programming to calculate the number of unique paths from the top-left corner to the bottom-right corner of a grid of size `m` by `n`. The function initializes a DP table `dp` and iterates through the grid to calculate the number of paths. Below the function, there are test cases for `(3, 7)`, `(3, 2)`, and `(7, 3)`. The output panel shows the results of these test cases: 28 for `(3, 7)`, 3 for `(3, 2)`, and 28 for `(7, 3)`. The execution was successful.

```
def unique_paths(m, n):
    dp = [[1] * n for _ in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]

# Test Cases
print("Test Case 1 (3x7):", unique_paths(3, 7))
print("Test Case 2 (3x2):", unique_paths(3, 2))
print("Test Case 3 (7x3):", unique_paths(7, 3))
```

Test Case 1 (3x7): 28
Test Case 2 (3x2): 3
Test Case 3 (7x3): 28
=== Code Execution Successful ===

20. Given an array of integers `nums`, return the number of good pairs. A pair (i, j) is called good if `nums[i] == nums[j]` and $i < j$.

Example 1:

Input: `nums = [1,2,3,1,1,3]`

Output: 4

Explanation: There are 4 good pairs $(0,3)$, $(0,4)$, $(3,4)$, $(2,5)$ 0-indexed.

Example 2:

Input: `nums = [1,1,1,1]`

Output: 6

Explanation: Each pair in the array are good.

The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for counting identical pairs in an array. The output panel on the right shows the results of three test cases and a success message.

```

def num_identical_pairs(nums):
    count = {}
    result = 0
    for num in nums:
        count[num] = count.get(num, 0) + 1

    for freq in count.values():
        result += freq * (freq - 1) // 2

    return result

# Test Cases
print("Test Case 1 [1,2,3,1,1,3]:",
      num_identical_pairs([1,2,3,1,1,3]))
print("Test Case 2 [1,1,1,1]:",
      num_identical_pairs([1,1,1,1]))
print("Test Case 3 [1,2,3]:",
      num_identical_pairs([1,2,3]))
  
```

Output:

```

Test Case 1 [1,2,3,1,1,3]: 4
Test Case 2 [1,1,1,1]: 6
Test Case 3 [1,2,3]: 0
=== Code Execution Successful ===
  
```

21. There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`. Return the city with the smallest number of cities that are reachable through some path and whose distance is at most `distanceThreshold`. If there are multiple such cities, return the city with the greatest number. Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

Example 1:

Input: $n = 4$, `edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]`,
`distanceThreshold = 4`

Output: 3

Explanation: The figure above describes the graph.

The neighboring cities at a `distanceThreshold = 4` for each city are:

City 0 -> [City 1, City 2]

City 1 -> [City 0, City 2, City 3]

City 2 -> [City 0, City 1, City 3]

City 3 -> [City 1, City 2]

Cities 0 and 3 have 2 neighboring cities at a distance Threshold = 4, but we have to return city 3 since it has the greatest number.

Example 2:

Input: $n = 5$, `edges = [[0,1,2],[0,4,8],[1,2,3],[1,4,2],[2,3,1],[3,4,1]]`,
`distance Threshold = 2`

Output: 0

Explanation: The figure above describes the graph.

The neighboring cities at a distance Threshold = 2 for each city are:

City 0 -> [City 1]

City 1 -> [City 0, City 4]

City 2 -> [City 3, City 4]

City 3 -> [City 2, City 4]

City 4 -> [City 1, City 2, City 3]

The city 0 has 1 neighboring city at a distanceThreshold = 2.

```
def find_the_city(n, edges, distanceThreshold):
    INF = float('inf')
    dist = [[INF] * n for _ in range(n)]

    for i in range(n):
        dist[i][i] = 0

    for u, v, w in edges:
        dist[u][v] = w
        dist[v][u] = w

    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    min_count = INF
    result_city = -1

    for city in range(n):
        count = 0
        for dst in range(n):
            if city != dst and dist[city][dst] <= distanceThreshold:
                count += 1

        if count <= min_count:
            min_count = count
            result_city = city

    return result_city
```

City: 3
Reachable Cities within threshold 4: 3
=== Code Execution Successful ===

22. You are given a network of n nodes, labeled from 1 to n . You are also given times, a list of travel times as directed edges $\text{times}[i] = (u_i, v_i, w_i)$, where u_i is the source node, v_i is the target node, and w_i is the time it takes for a signal to travel from source to target. We will send a signal from a given node k . Return the minimum time it takes for all the n nodes to receive the signal. If it is impossible for all the n nodes to receive the signal, return -1.

Example 1:

Input: $\text{times} = [[2,1,1],[2,3,1],[3,4,1]]$, $n = 4$, $k = 2$

Output: 2

Example 2:

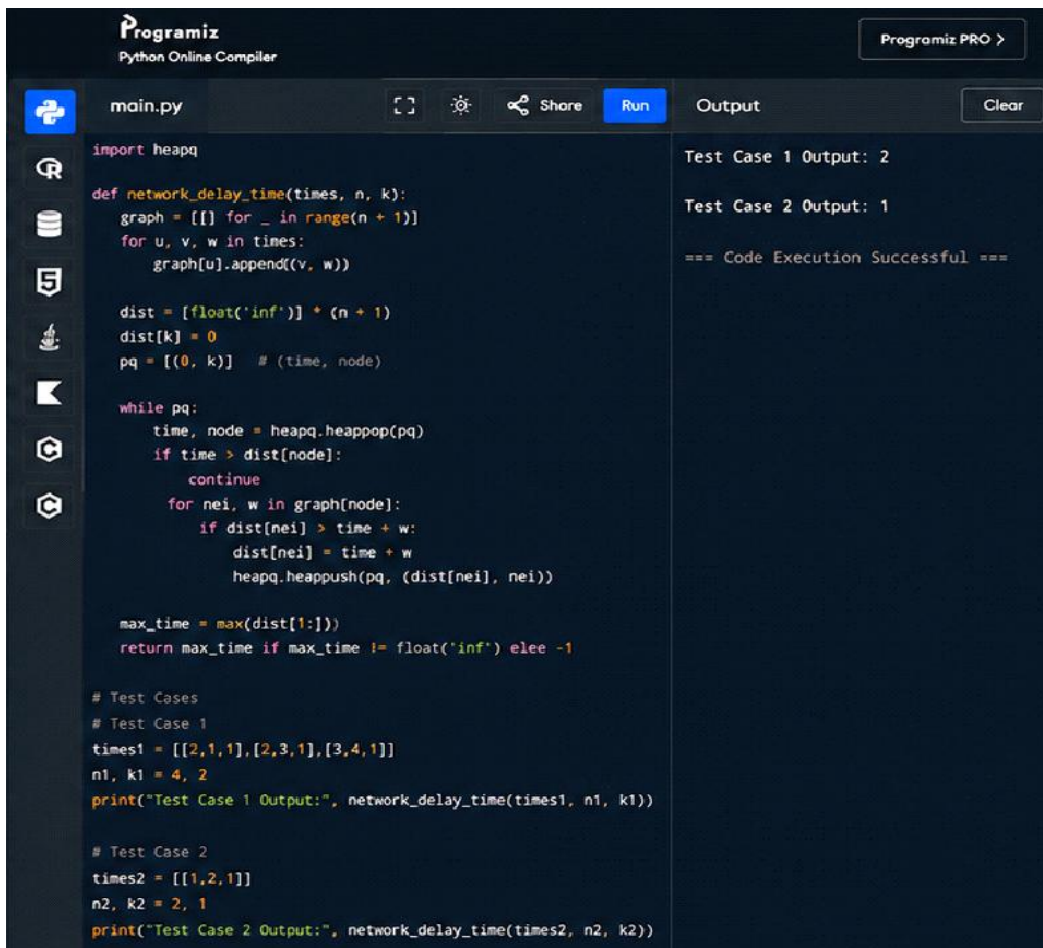
Input: $\text{times} = [[1,2,1]]$, $n = 2$, $k = 1$

Output: -1

Example 3:

Input: times = `[[1,2,1]]`, `n = 2`, `k = 2`

Output: -1



The screenshot shows the Programiz Python Online Compiler interface. The left sidebar contains icons for file management, search, and other tools. The main editor area displays a Python script named `main.py`. The script defines a function `network_delay_time` that takes `times`, `n`, and `k` as arguments. It constructs a graph with `n+1` nodes and edges defined by the `times` list. It then uses a priority queue (heap) to find the shortest path from node 0 to node `k`. The script includes test cases for `times1` and `times2`. The right panel shows the output of the script, which is "Test Case 1 Output: 2" and "Test Case 2 Output: 1", followed by a success message.

```
import heapq

def network_delay_time(times, n, k):
    graph = [[] for _ in range(n + 1)]
    for u, v, w in times:
        graph[u].append((v, w))

    dist = [float('inf')] * (n + 1)
    dist[k] = 0
    pq = [(0, k)]  # (time, node)

    while pq:
        time, node = heapq.heappop(pq)
        if time > dist[node]:
            continue
        for nei, w in graph[node]:
            if dist[nei] > time + w:
                dist[nei] = time + w
                heapq.heappush(pq, (dist[nei], nei))

    max_time = max(dist[1:])
    return max_time if max_time != float('inf') else -1

# Test Cases
# Test Case 1
times1 = [[2,1,1],[2,3,1],[3,4,1]]
n1, k1 = 4, 2
print("Test Case 1 Output:", network_delay_time(times1, n1, k1))

# Test Case 2
times2 = [[1,2,1]]
n2, k2 = 2, 1
print("Test Case 2 Output:", network_delay_time(times2, n2, k2))
```

Test Case 1 Output: 2

Test Case 2 Output: 1

=== Code Execution Successful ===